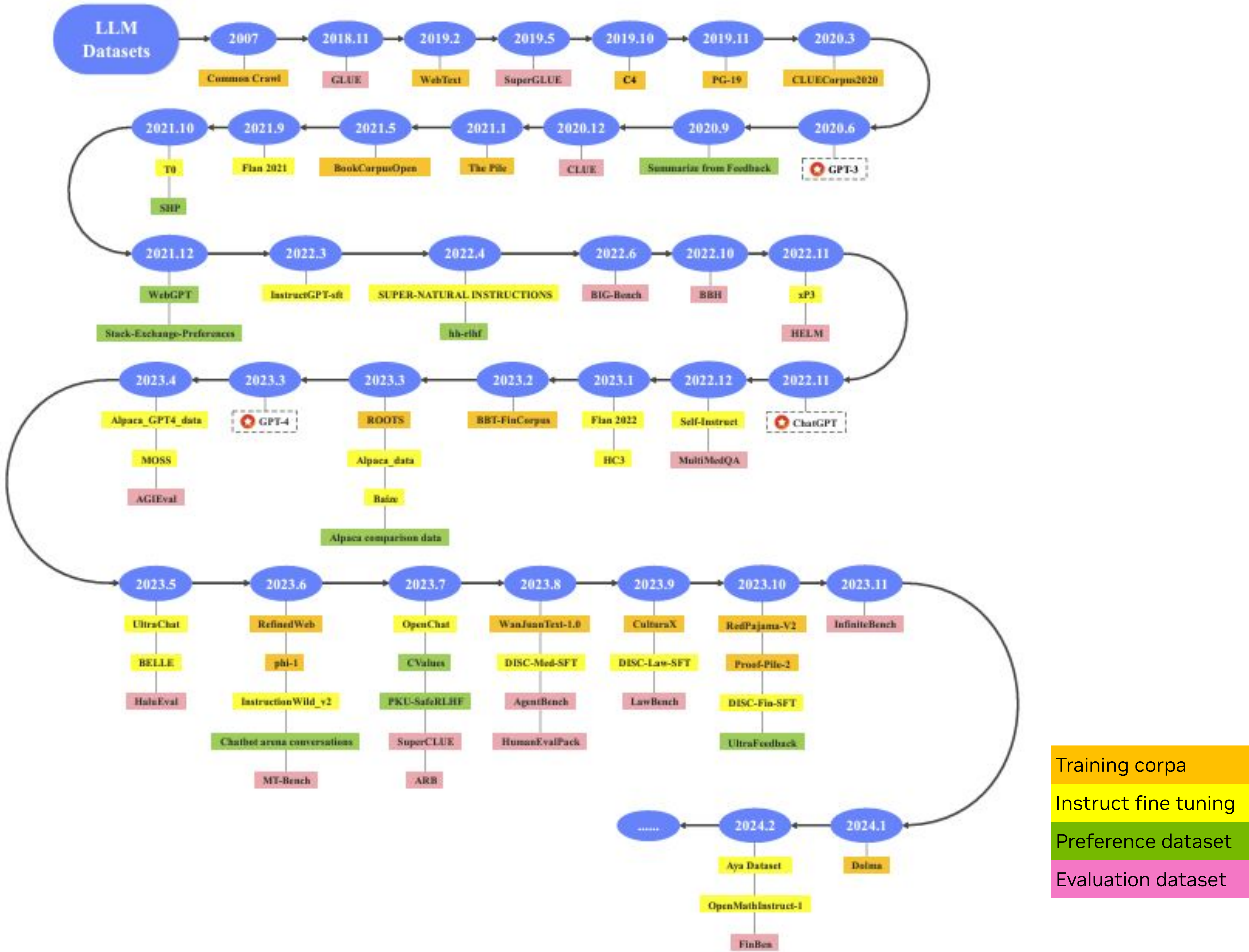


Unlocking AI Performance with NeMo Curator: Scalable Data Processing for LLMs

Allison Ding, Senior Developer Advocate - Data Science | SciPy July 2025

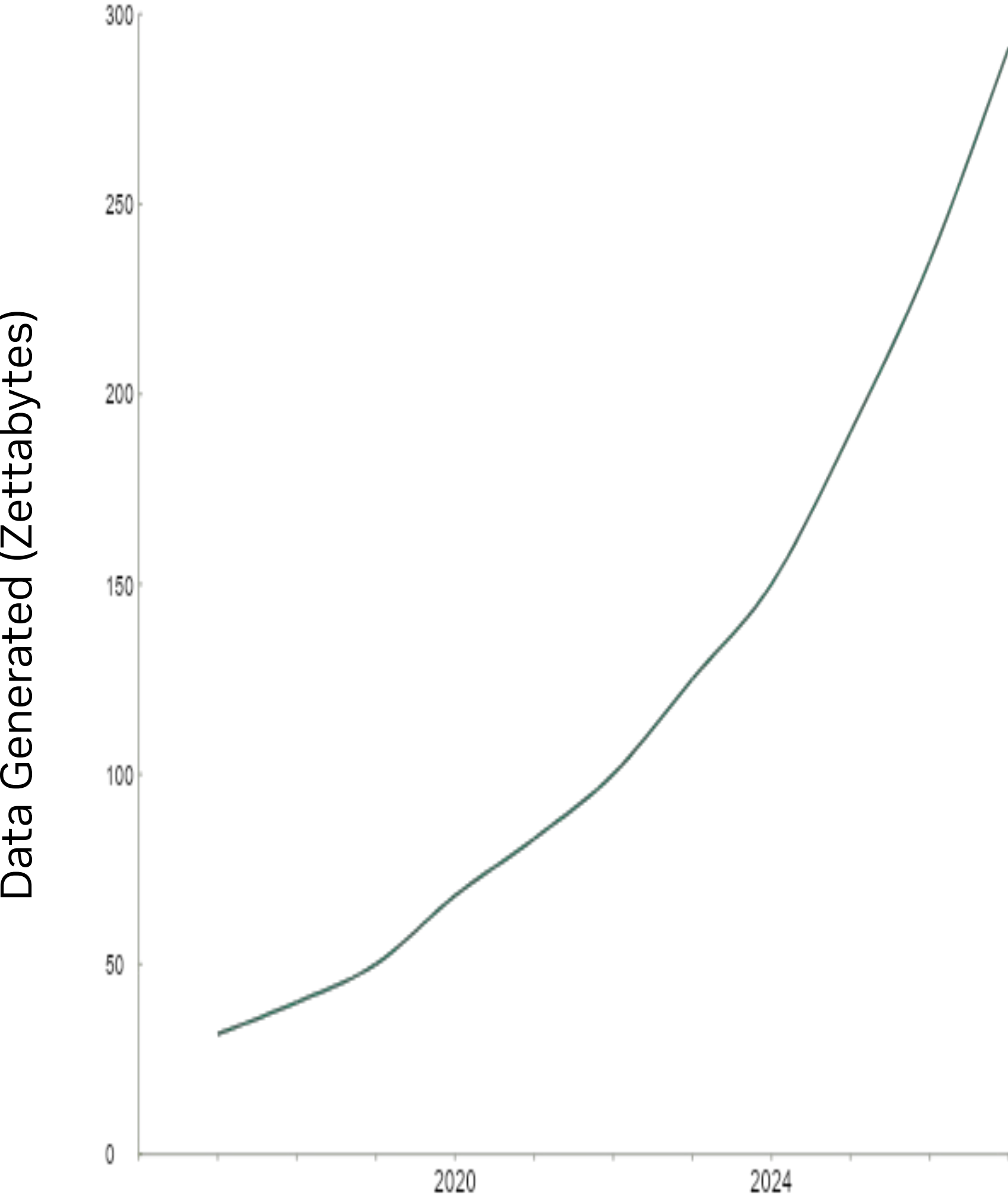
LLMs Are Trained on Internet Scale Data

Data generated is growing exponentially

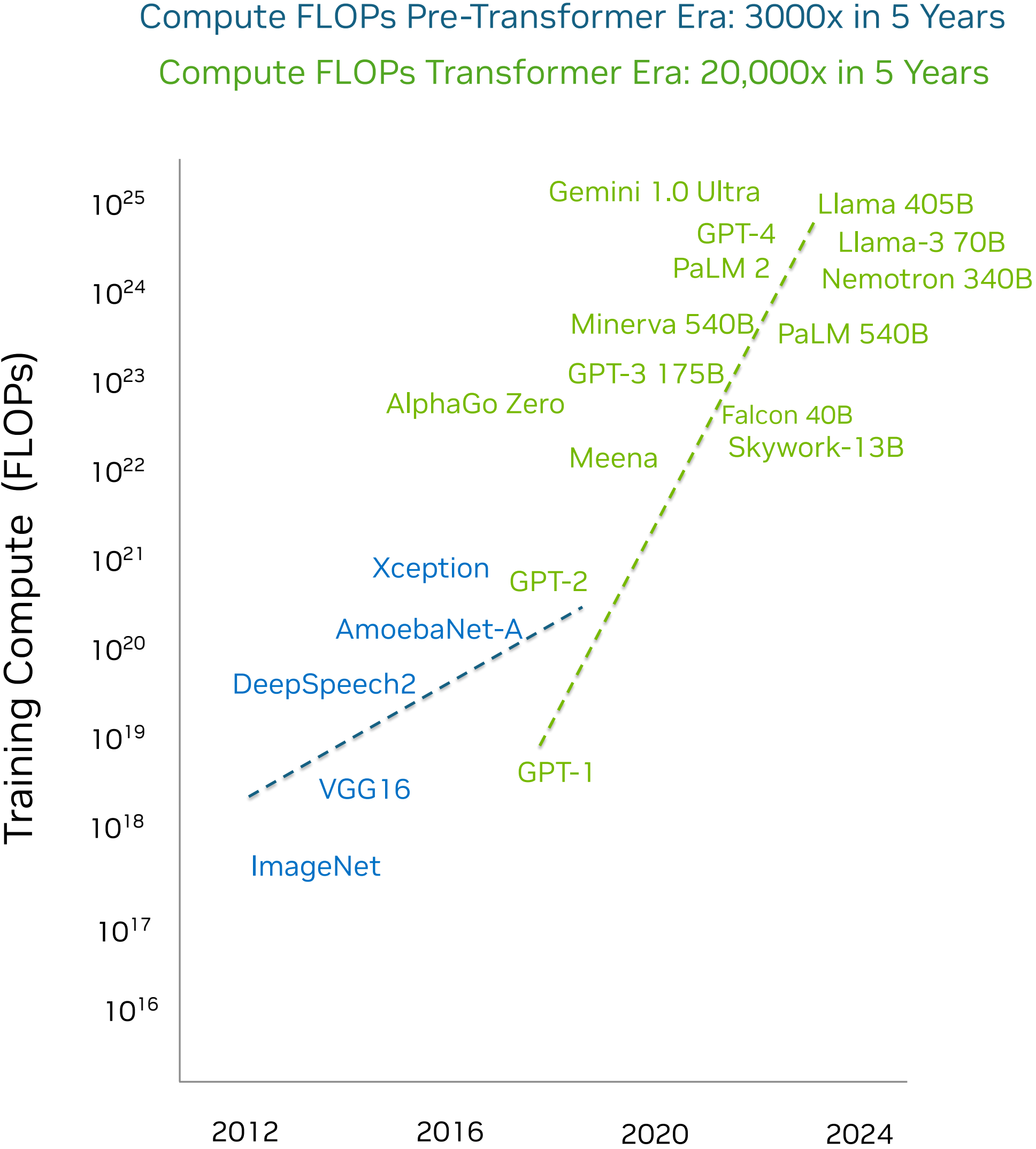


Data Processing for LLMs Needs Accelerated Computing

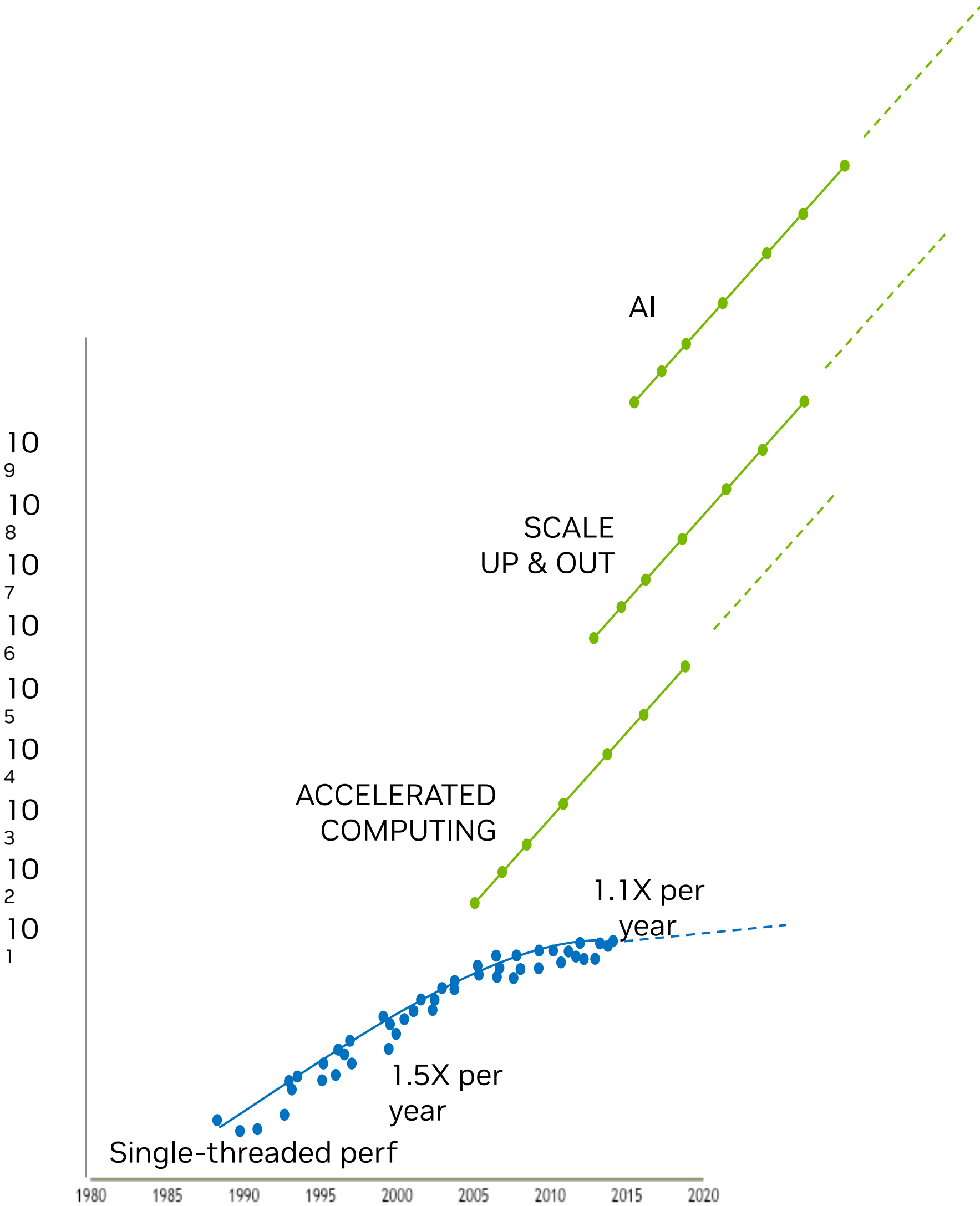
Petabytes of Data Generated Yearly



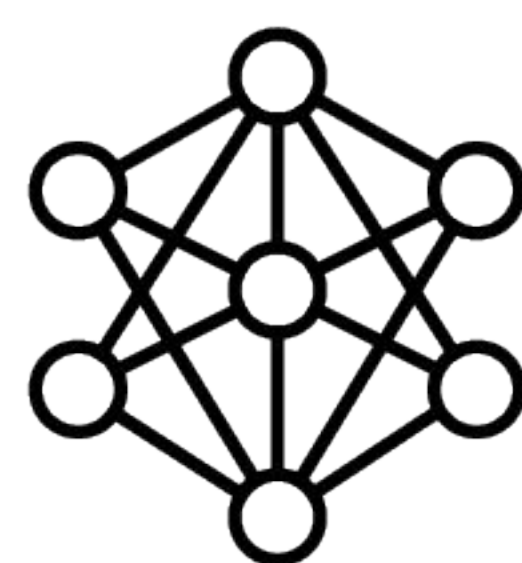
LLMs Trained on Internet Scale Data



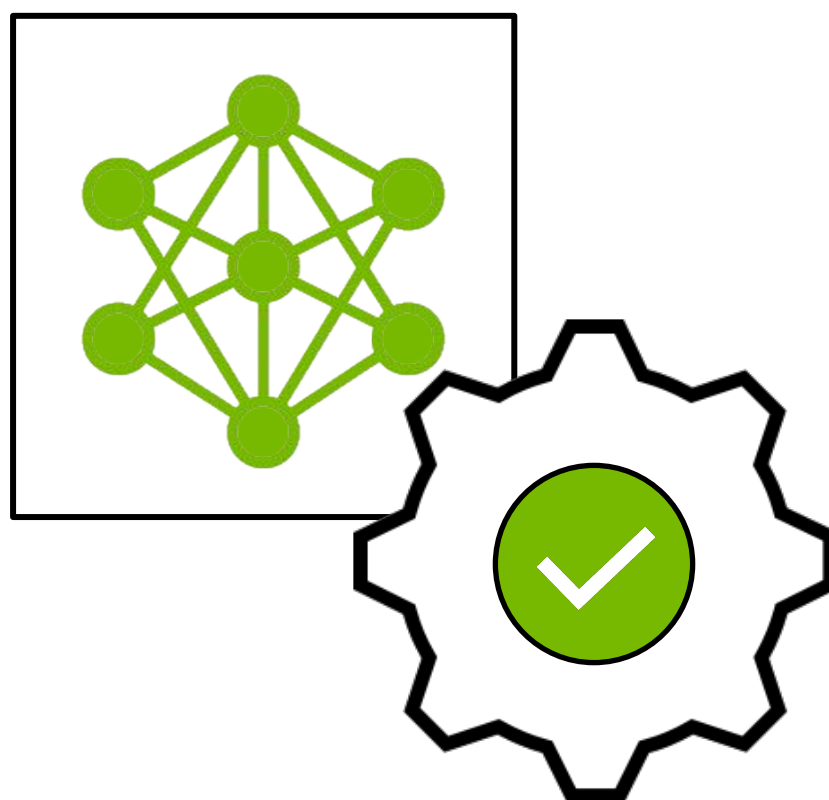
Moore's Law Has Ended



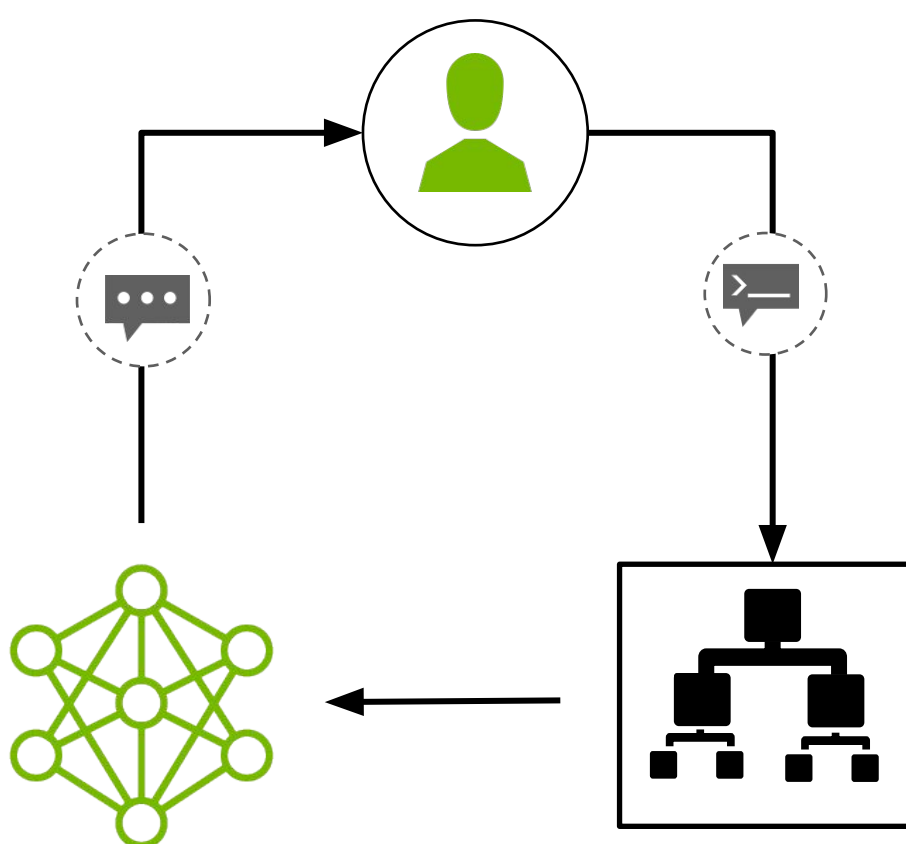
Data Processing for Different LLM Needs



Training Foundation Model



Fine-Tuning Foundation Model

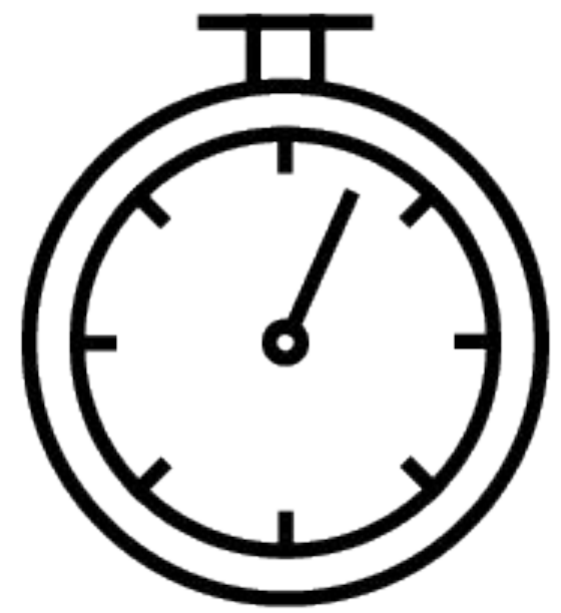


Retrieval Augmented Generation (RAG)

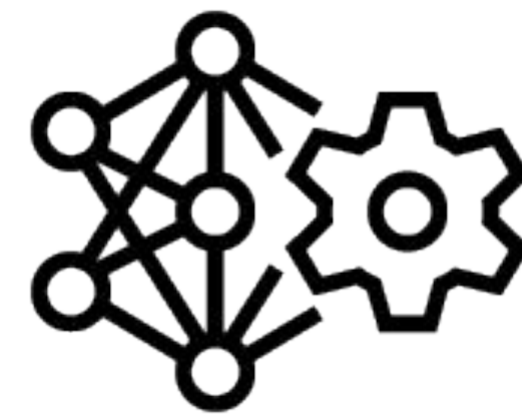
Data Size	TB and PB	GBs	GBs
Compute Scale	Supercomputer	Single-node	Single GPU
Frequency	One-time	Iterative	Iterative & Continuous

Challenges with Existing Solutions for Training Foundation Models

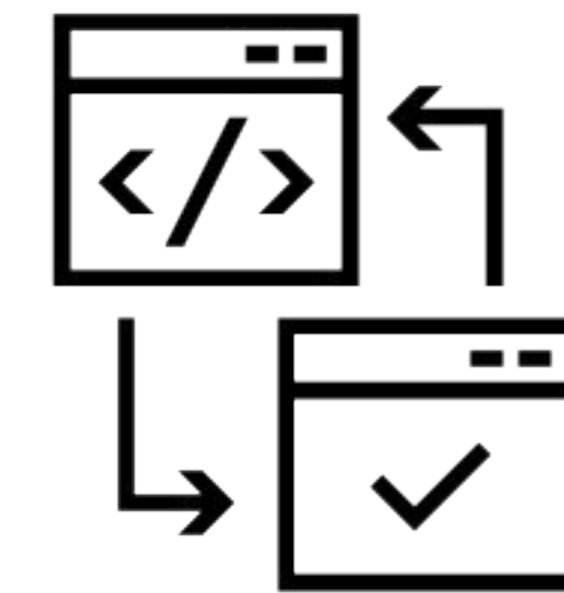
Inefficiencies lead to higher TCO and slower time to market



Longer Processing Time



Un-optimized Models



Un-optimized Pipelines



Knowledge & Expertise

NeMo Curator - Overview

Scalable, configurable pipelines to curate text, image and video datasets lead to more accurate applications

Higher Accuracy



Improve accuracy with less data and less training compute

Faster Processing



GPU acceleration with RAPIDS for **Dedupe (Exact, Fuzzy, Semantic)** and **Quality Classifier Models**

Scalability



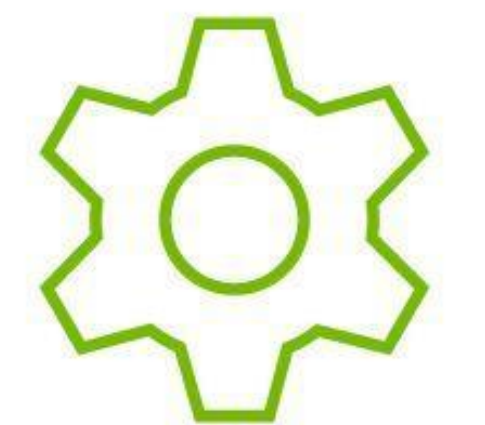
Up to **100+ PB data** by scaling across multiple nodes

Classifier Models



State-of-the-Art quality classifier NIM microservice for safety, content, and diversity

Deploy anywhere



Python APIs in a customizable and modular OSS library, runnable across CSP and On-prem

[GitHub](#)

[NeMo framework container](#)

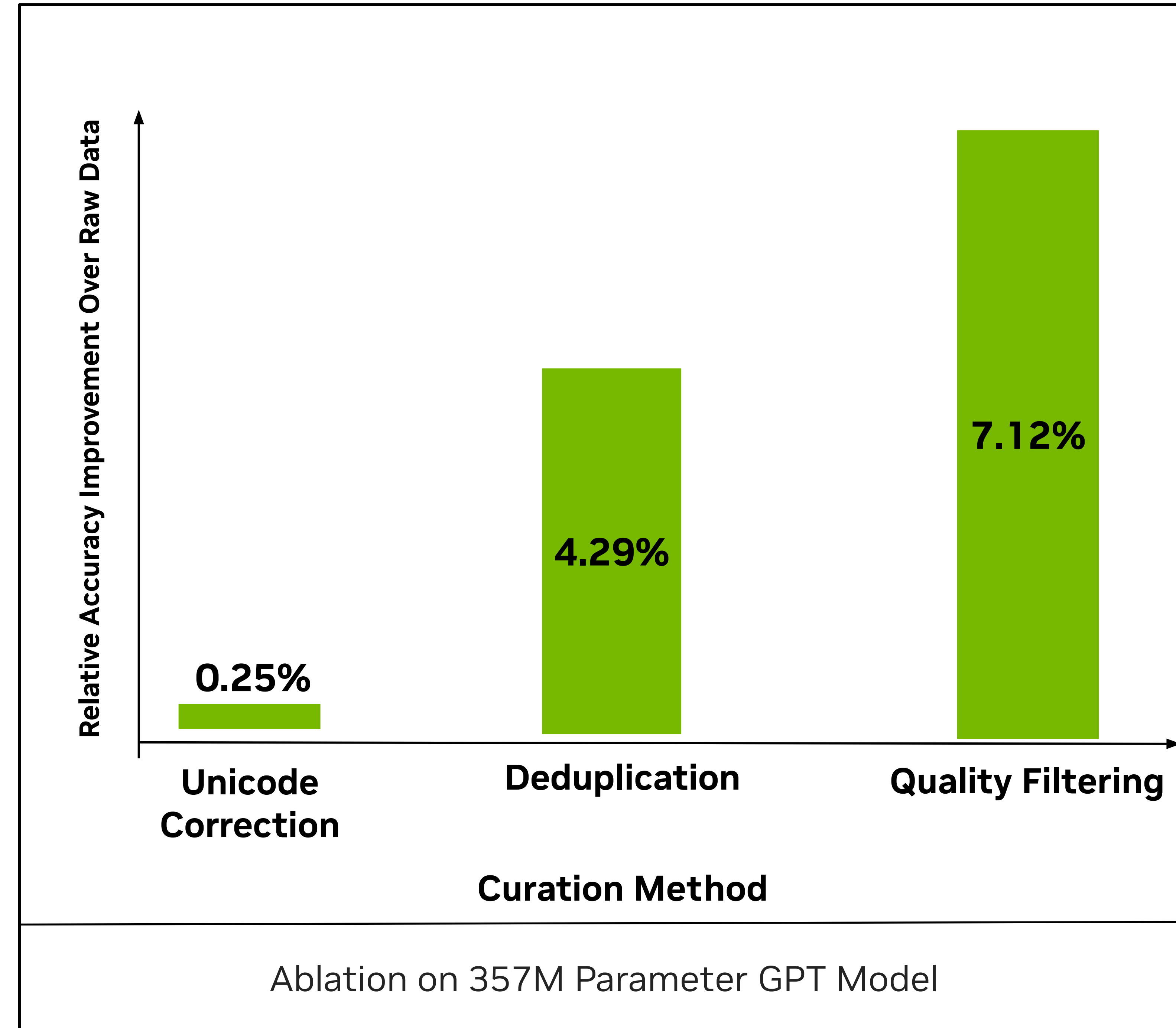
[PyPI](#)

Microservice
(coming soon)

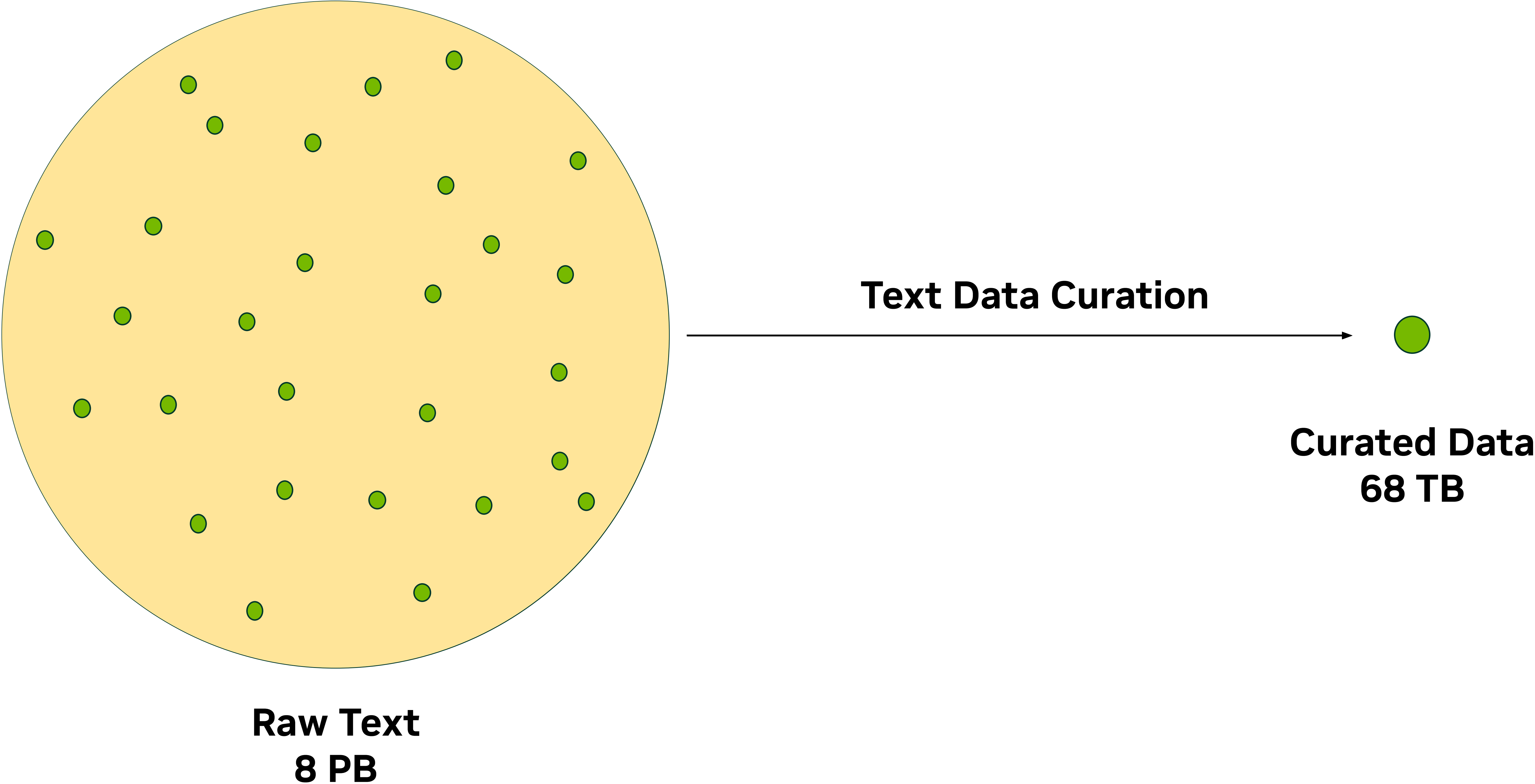
High Quality Data Processing Maximizes Model Performance

Data Curation helps build SOTA Models

LLM Accuracy Improvement on Curated Data

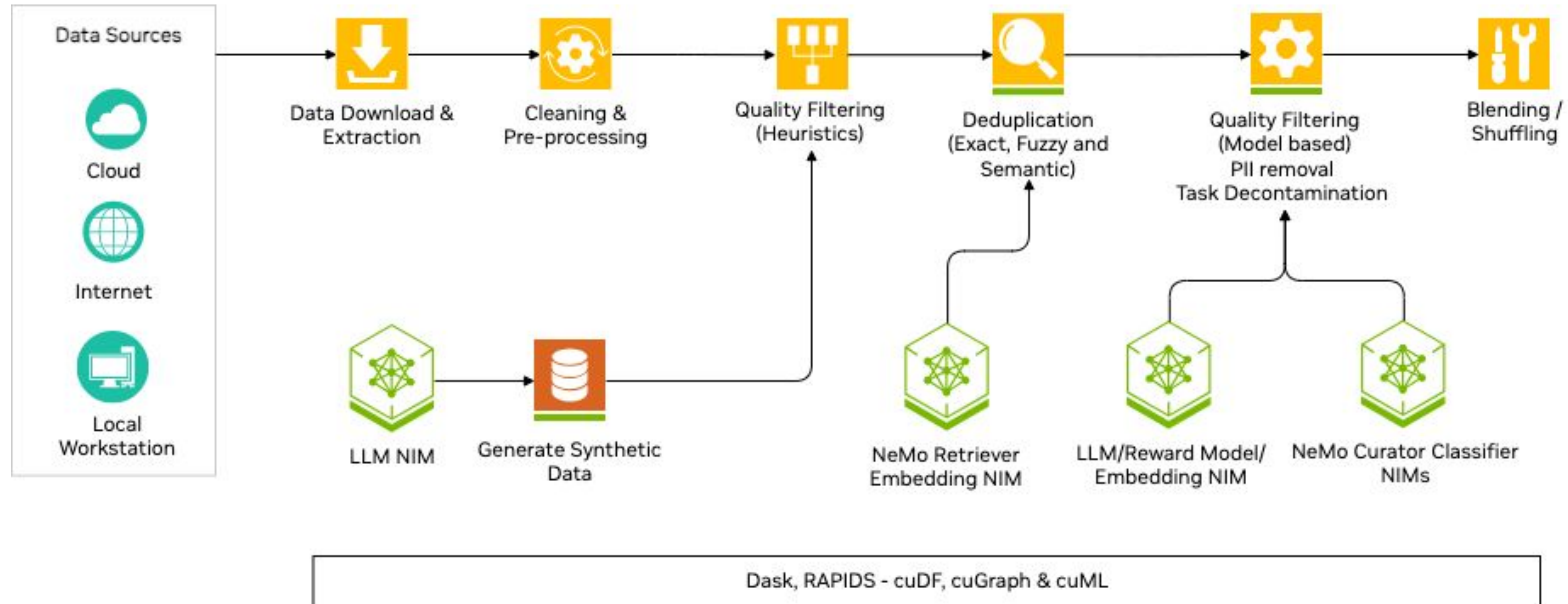


Only 1% of Raw Text Data is Curated to Train Foundation Models



Introducing: NeMo Curator for Text Processing

Easily integrate different features into your existing pipelines with Python APIs



 GPU accelerated

[GitHub](#)

[NeMo framework container](#)

[PyPI](#)

NeMo Curator: Features to Train Foundation Models

Achieve higher accuracy with a variety of GPU-accelerated features



Synthetic Data Generation

- ❑ **Pre-built pipelines** - for tasks like prompt generation, dialogue generation, and entity classification
- ❑ **Modular** - Easily integrate NeMo Curator's features into your existing pipelines
- ❑ **OpenAI API compatible** - Integrate custom Instruct and Reward models



Deduplication & Classification

- ❑ **Lexical Deduplication** – Identical (Exact) or near identical (Fuzzy)
- ❑ **Semantic Deduplication** – focuses on the meaning rather than the exact text
- ❑ **Classifier Models** - State-of-the-art open models to either enrich or filter your data.

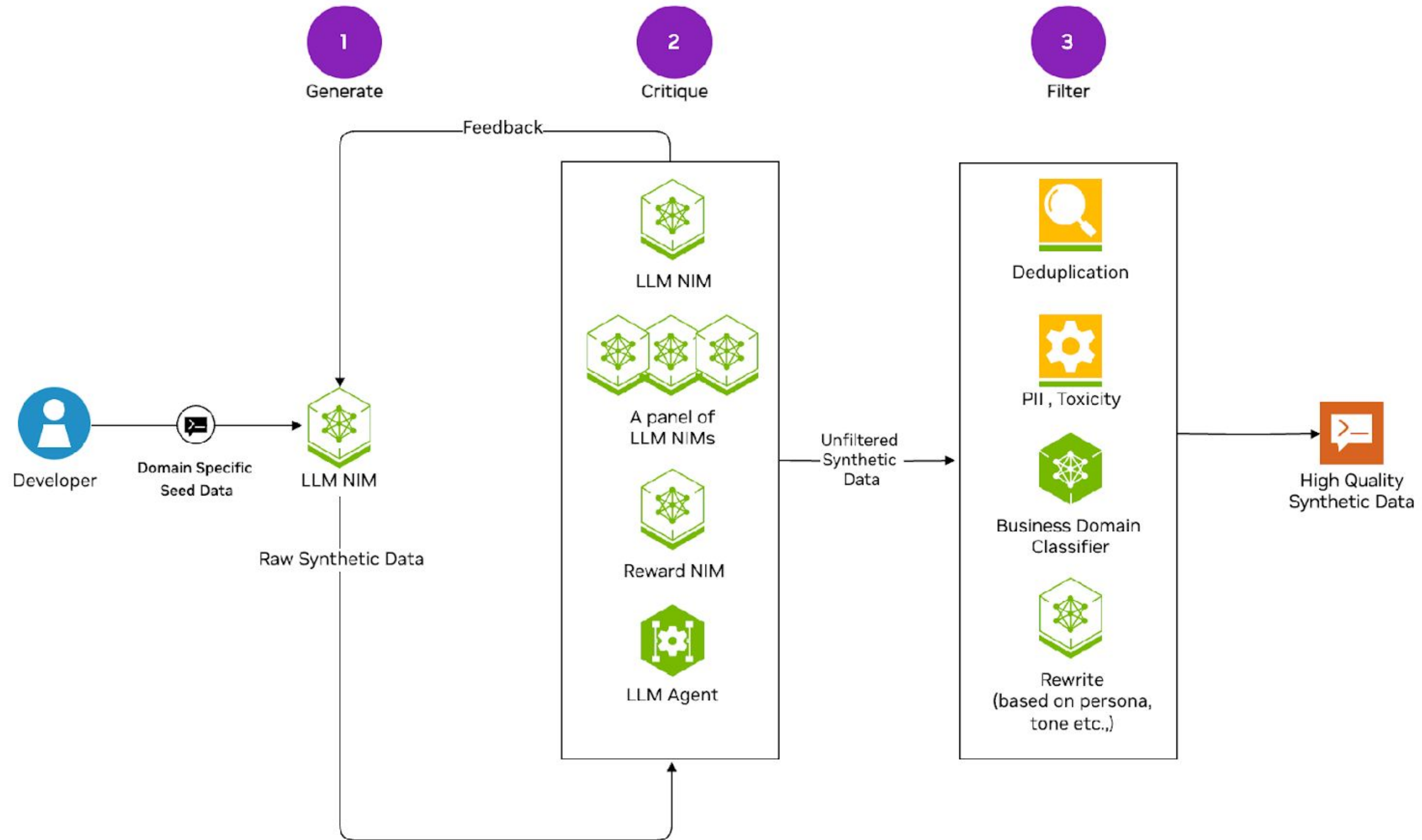


GPU Acceleration with RAPIDS

- ❑ **cuDF** - for deduplication & classifier models
- ❑ **cuML** - for K-means clustering in semantic deduplication
- ❑ **cuGraph** – for fuzzy deduplication

Synthetic Data Generation

Easily get started with pre-built pipelines or integrate various features into your existing workflows



Synthetic Data Generation

Easily get started with pre-built pipelines or integrate various features into your existing workflows

Pre-built Pipelines

- Prompt generation (open q/a, closed q/a, writing, math/coding)
- Synthetic two-turn prompt generation
 - Dialogue generation
 - Entity classification

Llama 3.1 Nemotron 70B/340B Reward Model

- Benchmark-Topping Performance
- Single scalar scoring for human preferences
- Permissive license for commercial use

Tooling Support

- Integrate into existing pipelines
- Bring your custom Instruct/Reward Model
- Supports different filtering techniques

Synthetic Data Generation: Example

Use prompts to synthetically generate the QnAs and the reward score

```
model = "nvdev/nvidia/llama-3.1-nemotron-70b-instruct"

question_lst = []

for i in range(len(ragas)):
    closed_qa_responses = client.query_model(
        model=model,
        messages = [
            {
                "role": "user",
                "content": f"Generate a single, concise question similar to '{ragas.question[i]}' based on the provided context.\n"
                           f"Do NOT include any explanations, rationale, or additional text in your response—just the question itself:\n"
                           f"{ragas.contexts[i]}"
            }
        ],
        temperature = 0.2,
        top_p = 0.7,
        max_tokens = 1024,
    )

    question = closed_qa_responses[0]
    question_lst.append(question)
```

```
ragas['sdg_nemotron_q'] = question_lst
```

✓ 8.7s

```
ragas['sdg_nemotron_q']
```

✓ 0.0s

```
0  What is the significance of including a module...
1  How does the task execution stage in HuggingGP...
2  What are the primary technical hurdles encount...
3  How does Algorithm Distillation (AD) leverage ...
4  What are the core architectural elements and o...
5  How do consistent naming conventions, intra-fi...
6  How does API-Bank assess LLMs' decision-making...
7  How does a package.json fit into defining a No...
8  How does API-Bank assess LLMs' utilization of ...
9  How do finite context lengths and long-term pl...
Name: sdg_nemotron_q, dtype: object
```

```
reward_lst = []

model = "nvdev/nvidia/llama-3.1-nemotron-70b-reward"

for i in range(len(ragas)):
    messages = [
        {
            "role": "user",
            "content": f""
                       Provide a concise and well-reasoned answer to the following question:
                       "{ragas.sdg_nemotron_q[i]}"
                       Use the context below to ensure accuracy and clarity:
                       {ragas.contexts[i]}
                       ""
        },
        {
            "role": "assistant",
            "content": ragas.sdg_nemotron_a[i],
        },
    ]

    rewards = client.query_reward_model(messages=messages, model=model)
    rewards = float(rewards)

    reward_lst.append(rewards)
```

```
ragas['sdg_nemotron_reward'] = reward_lst
```

✓ 3.1s

```
ragas['sdg_nemotron_reward']
```

✓ 0.0s

```
0  -13.93750
1  -4.09375
2  -8.68750
3  -8.06250
4  -5.15625
5  -15.75000
6  -8.18750
7  -9.31250
8  -7.56250
9  -11.06250
Name: sdg_nemotron_reward, dtype: float64
```


Synthetic Data Generation: Example

Build an SDG pipeline to generate the data

main.py

```
14
15 > import argparse--
38
39 SCRIPT_DIR_PATH = os.path.dirname(os.path.abspath(__file__))
40 DATA_DIR = os.path.join(SCRIPT_DIR_PATH, "data")
41 TEMP_DIR = os.path.join(SCRIPT_DIR_PATH, "_temp")
42 CONFIG_DIR = os.path.join(SCRIPT_DIR_PATH, "config")
43 DATASET_URL = "https://huggingface.co/datasets/ymoslem/Law-StackExchange/resolve/main/law-stackexchange-questions-answers.json"
44
45
46 > def pre_imports():--
48
49
50 > def random_split_rows(rows: List[Any], train_ratio: float, val_ratio: float, seed=42):--
72
73
74 > def download_and_convert_to_jsonl() -> str:--
114
115
116 > def semantic_dedupe(dataset):--
144
145
146 > def run_curation_pipeline(--
237
238
239 > def run_pipeline(args, jsonl_fp):--
364
365
366 > def main():--
432
433
434 > if __name__ == "__main__":--
```

syn_gen.py

```
14
15 > import asyncio--
26
27 > PROMPT_GENERATE_QUESTIONS_FROM_ANSWER = """TEXT: --
36
37 > PROMPT_PARAPHRASE_TEXT = """TEXT: --
46
47
48 class SyntheticGenerator:
49
50 > def __init__(--
81
82 > def run(--
106
107 > def _split_sdg_responses(self, sdg_response: str) -> List[str]:--
120
121 > def _write_all_to_file(self, gen_entries, out_fp: str):--
182
183 > async def _prompt_model(--
252
253 > async def _synthesize_from_source(--
```

multiple rounds of SDG

```

  data
  curated
  final
  {} law-qa-test.jsonl
  {} law-qa-train.jsonl
  {} law-qa-val.jsonl
  round-1
  {} law-qa-train-synth-round-1.jsonl
  {} law-qa-train.jsonl
  round-2
  round-3
  round-4
  round-5
  raw
  downloads
  {} law-stackexchange-questions-answers.json
  splits
  {} law-qa-test.jsonl
  {} law-qa-train.jsonl
  {} law-qa-val.jsonl
```

one example on SDG curated result

```
{
  "id": "appliance-qa-B00074TB9U-synth-0",
  "asin": "B00074TB9U-synth-0",
  "question": "How effective is a ductless installation in eliminating cooking smells?",
  "answer": "The vent has two speeds; the lower one is suitable for regular air removal, while the higher speed is necessary for heavy-duty cooking. The higher speed functions well.",
  "questionType": "yes/no",
  "score": -1,
  "helpfulness": 0.6015625,
  "correctness": 0.388671875,
  "coherence": 3.078125,
  "complexity": 0.8671875,
  "verbosity": 0.60546875
}
```


Deduplication and Filtering

Scale across multi-node, multi-GPU setups, eliminating the need for iterative CPU processing

- Supports lexical and semantic deduplication for document processing
- Scales on multi node, multi-GPU by leveraging RAPIDS
- Scale to 100+ TB of data

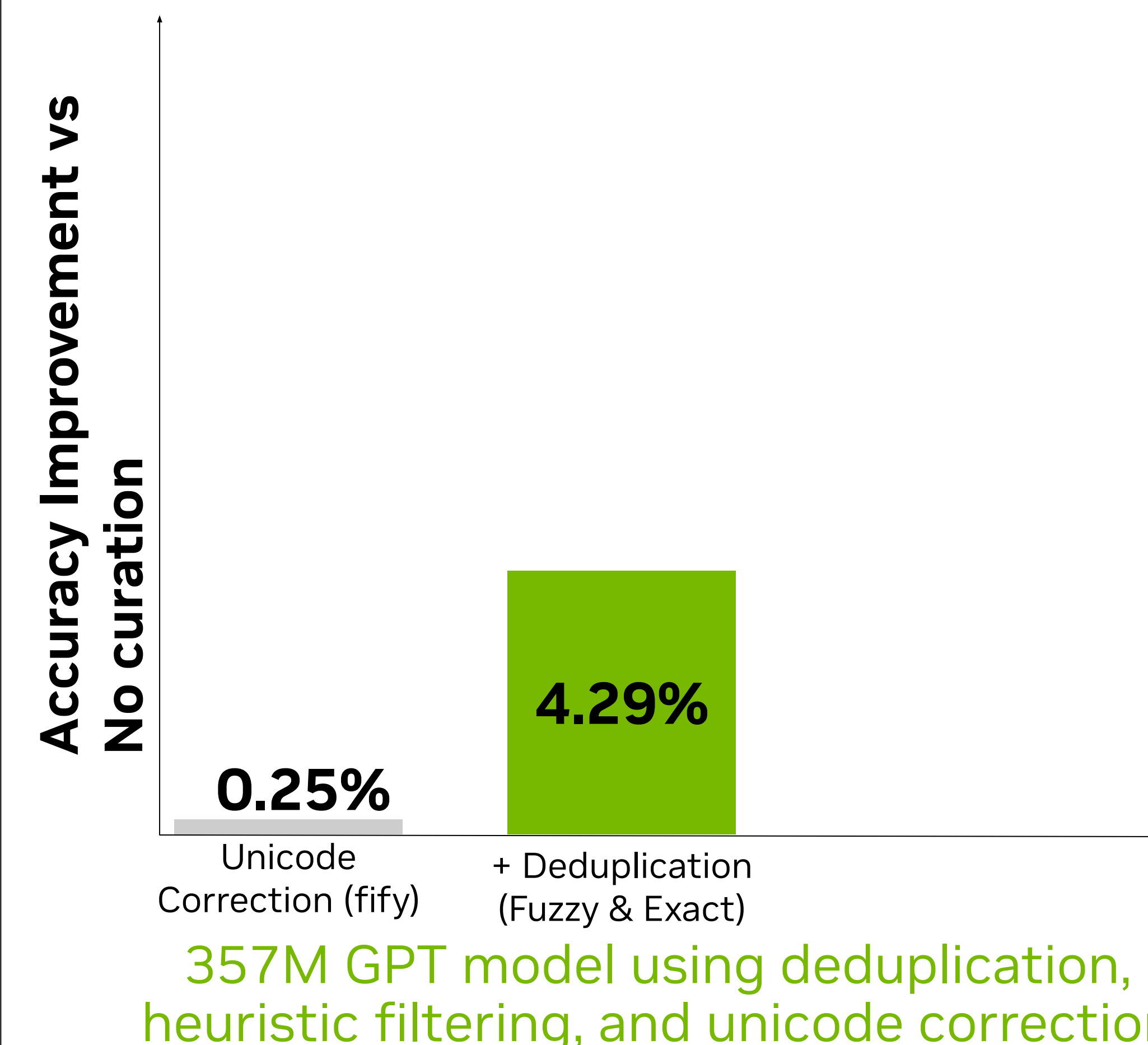
Lexical Deduplication

- Utilizes text similarity to discover duplication
- **Exact deduplication**
 - Hashing based matching for each document
- **Fuzzy deduplication**
 - Minhash, Bucketization and Clustering

Semantic Deduplication

- Utilizes meaning of text to discover duplication
- Leverages embedding models to identify semantic documents
- Modularity to use custom embedding model

Acceleration



Semantic Deduplication: Example

Remove redundant data by identifying and eliminating semantically similar data points

sem_dedup_config.yaml

```
# Configuration file for semantic dedup
cache_dir: "semdedup_cache"
num_files: -1

# Embeddings configuration
embeddings_save_loc: "embeddings"
embedding_model_name_or_path: "sentence-transformers/all-MiniLM-L6-v2"
embedding_batch_size: 128

# Clustering configuration
clustering_save_loc: "clustering_results"
n_clusters: 1000
seed: 1234
max_iter: 100
kmeans_with_cos_dist: false

# Semdedup configuration
which_to_keep: "hard"
largest_cluster_size_to_process: 100000
sim_metric: "cosine"

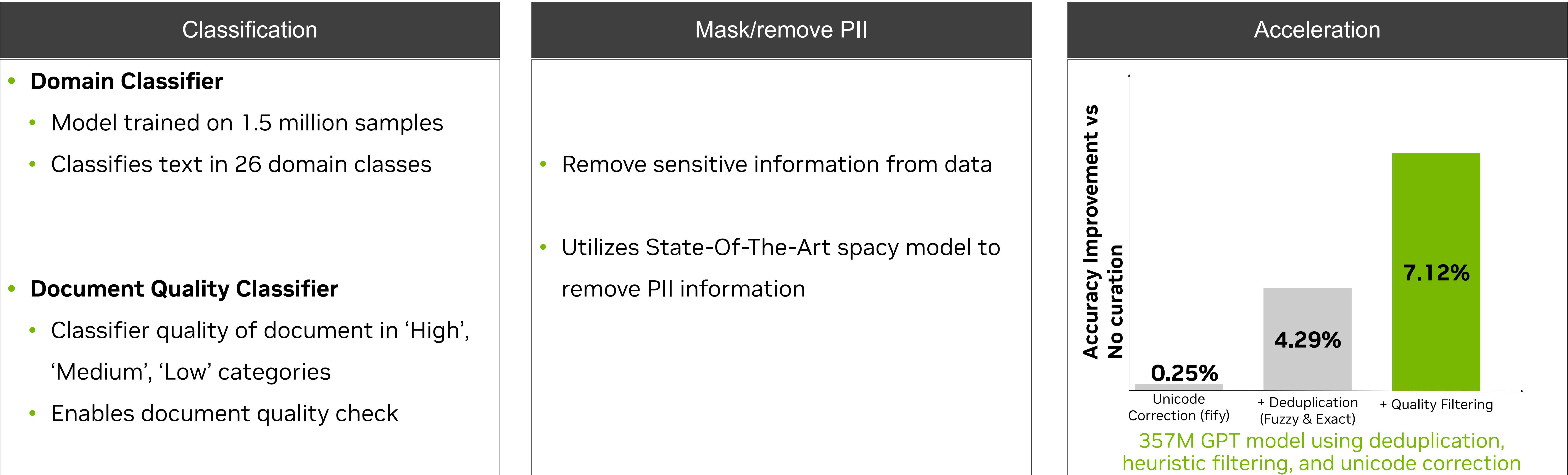
# Extract dedup configuration
eps_thresholds:
  - 0.01
  - 0.001

# Which threshold to use for extracting deduped data
eps_to_extract: 0.01
```


Classification and PII

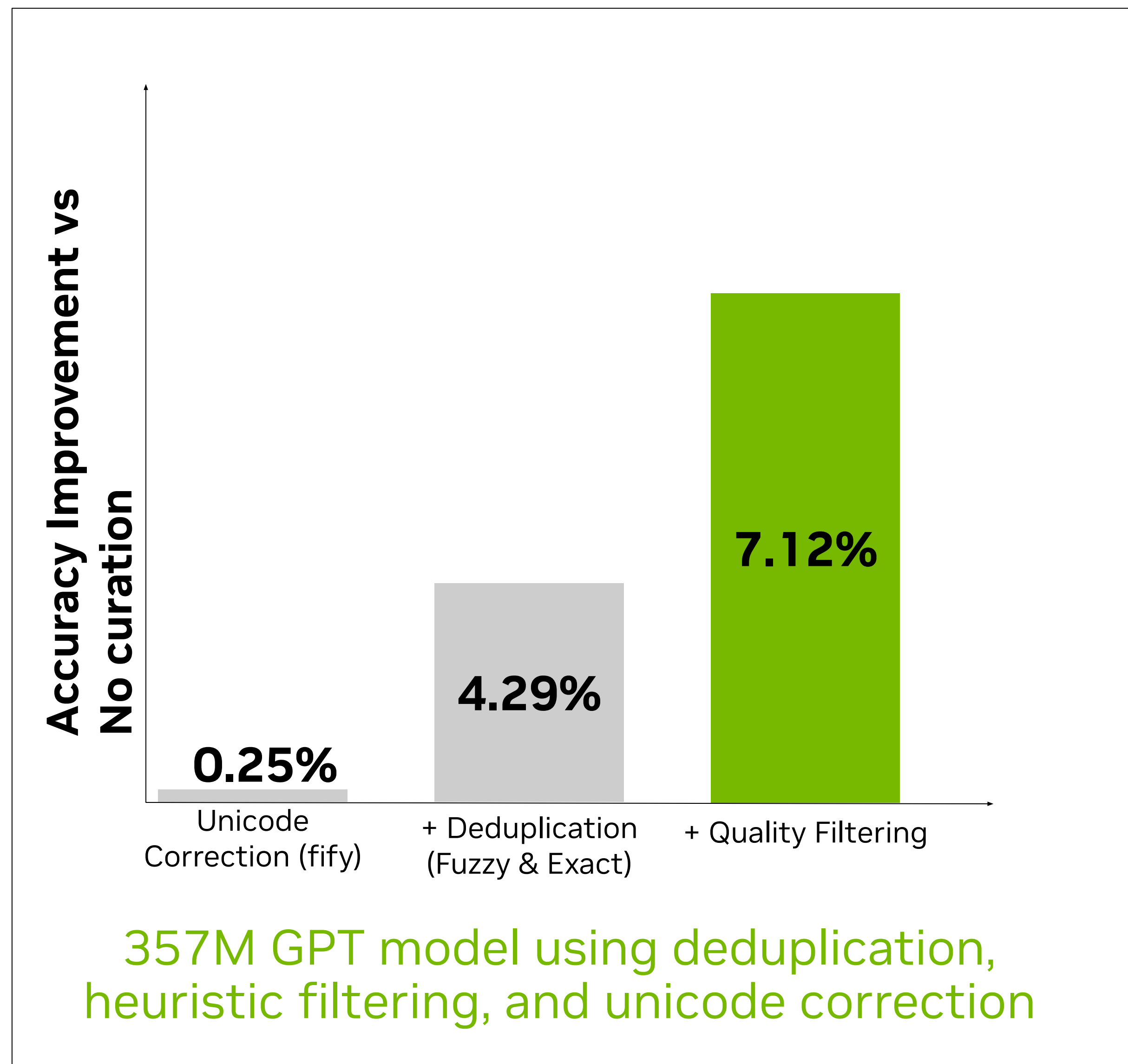
Create high-quality data blends with RAPIDS accelerated inference

- Accelerated inference through distributed classification model and intelligent batching
- Redact/remove Personally Identifiable information (PII) using SOTA model

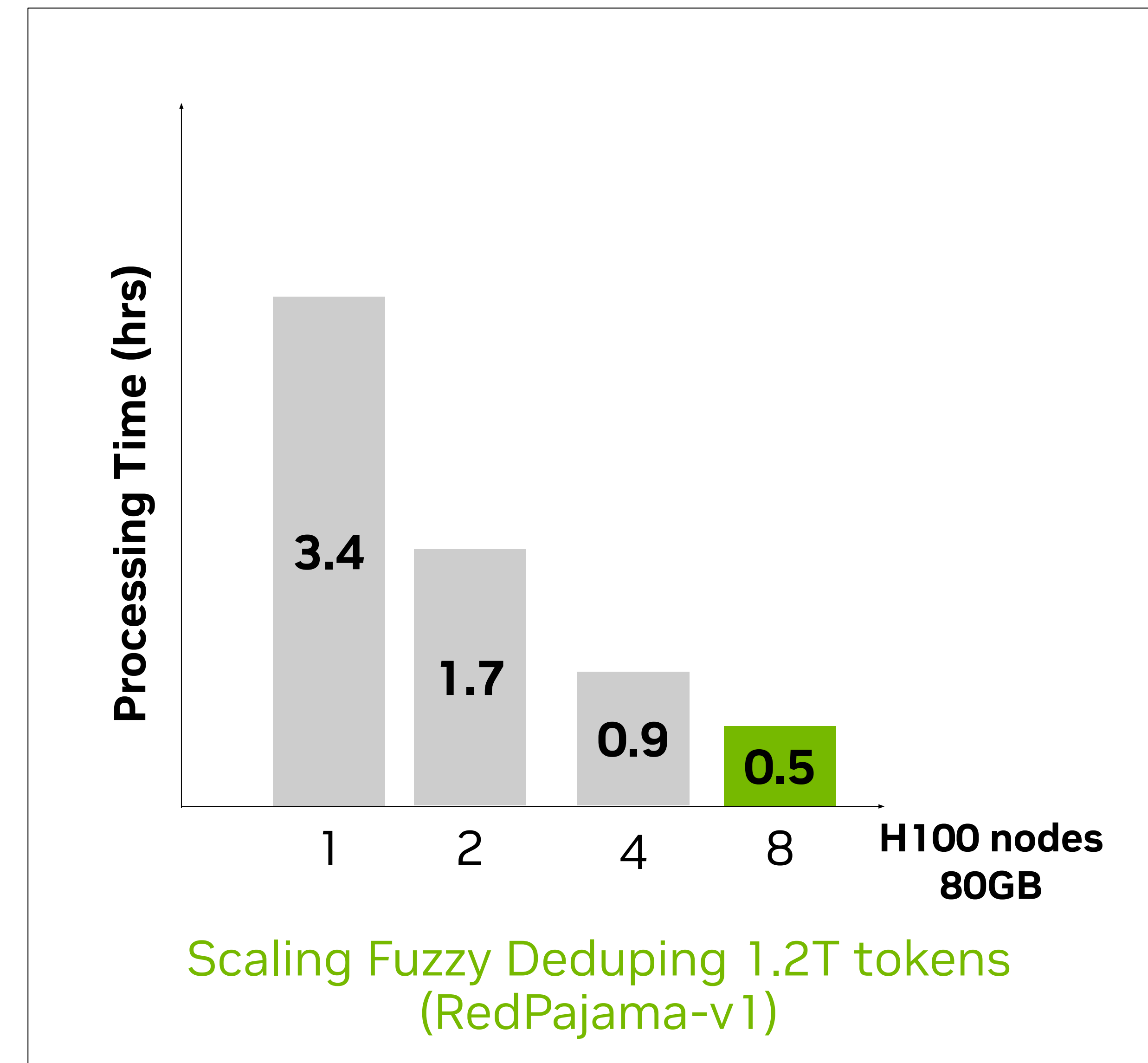


Accelerated Data Processing Maximizes LLM Performance & Scale

Nemo Curator: Up to 7% better Accuracy for LLM Downstream Tasks



Scale to 100+ TB of data



NeMo Curator: Example

```
# Download your dataset
dataset = download_common_crawl("/datasets/common_crawl/", "2021-04", "2021-10", url_limit=10)
# Build your pipeline
curation_pipeline = Sequential([
    # Fix unicode
    Modify(UnicodeReformatter()),
    # Discard short records
    ScoreFilter(WordCountFilter(min_words=80)),
    # Discard low-quality records
    ScoreFilter(FastTextQualityFilter(model_path="model.bin")),
    # Discard records from the evaluation metrics to prevent test set leakage.
    TaskDecontamination([Winogrande(), Squad(), TriviaQA()])
])
# Execute the pipeline on your dataset
curated_dataset = curation_pipeline(dataset)
```

[GitHub](#)[NeMo framework container](#)[PyPI](#)

NeMo Curator - Resources

Getting Started

- [NeMo Framework Container](#)
- [GitHub](#)
- [PyPI](#)
- [Installation Guide](#)
- [User Guide/ Docs](#)
- [API Docs](#)
- [Classifier Models](#)
- [Examples](#)
- [Best Practices](#)
- [Bugs](#)
- [Discussions](#)

Tutorials & Blogs

Pre-training / DAPT

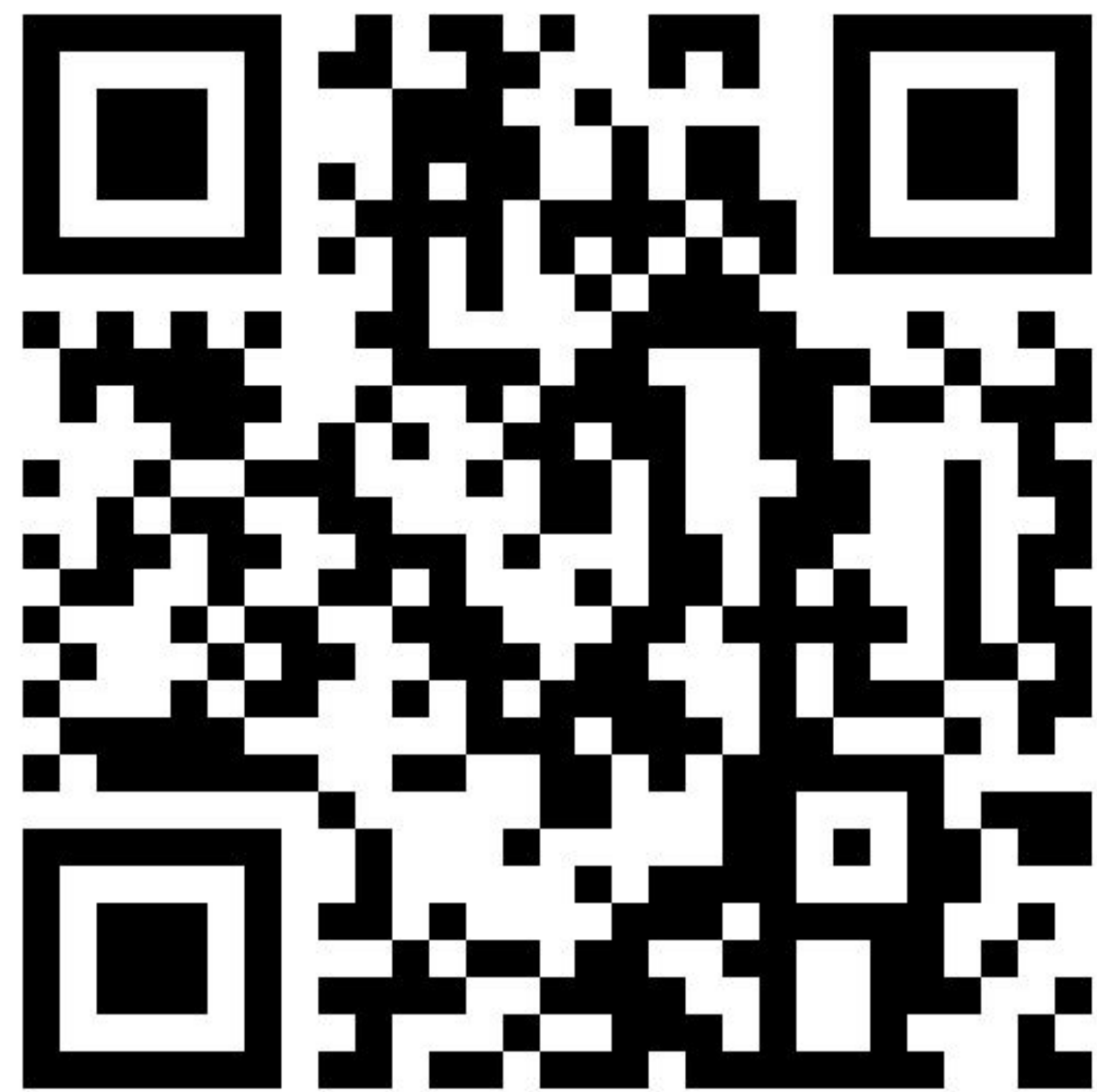
- [Curating data for LLM training \(Blog\)](#)
- [Curating non-English data \(Blog\)](#)
- [How-to run classifier models](#)
- [All blogs](#)

Fine-tuning

- [Curating data for PEFT \(Blog\)](#)
- [Curating synthetic data for PEFT](#)
- [SDG using Llama 3.1 405B & Nemotron 4-340B](#)
- [SDG using Nemotron 4-340B](#)

Developer Tools and Resources

Accelerate innovation and growth



Learn more: developer.nvidia.com

Individuals

Software

100s of APIs, models, SDKs, microservices, and early access to NVIDIA tech

Training

Hands-on self-paced courses, instructor-led workshops, and certifications

GPU Sandbox

Approval basis, multi-GPU and multi-node

Learning

Tutorials, self-paced courses, blogs, documentation, code samples

Community

Dedicated developer forums, meetups, hackathons

Ecosystem

GTC, NVIDIA Partner Network

Organizations

Startups

Cloud credits, engineering resources, technology discounts, exposure to VCs

Venture Capital

Deal flow and portfolio support for Venture Capital firms

Higher Education

Teaching kits, training, curriculum co-development, grants

ISVs and SIs

Engineering guidance, discounts, marketing opportunities

Research

Grant programs, collaboration opportunities

Enterprises

Tailored developer training, skills certification, technical support

Learn New Skills for Free

Access essential technical training

Get a free technical course (worth up to \$90). Scan the QR code, enroll in your chosen course today, and complete within the next 6 months.

Courses include:

- Accelerating End-to-End Data Science Workflows
- Best Practices in Feature Engineering for Tabular Data With GPU Acceleration
- Augment your LLM Using Retrieval Augmented Generation
- Building LLM Applications With Prompt Engineering
- Building RAG Agents with LLMs
- Fundamentals of Accelerated Computing with CUDA Python
- Generative AI with Diffusion Models
- Getting Started with Deep Learning
- Introduction to Deploying RAG Pipelines for Production at Scale
- Introduction to NVIDIA NIM™ Microservices

Scan the QR code to access the full course list and redeem your free training. <https://sp-events.courses.nvidia.com/AIDaysEU>





Thank You